COMP 512 - Programming Assignment 3

Karl Wehbe, Ion Basha

May 22, 2026

1 Introduction

In this project, we implemented a load balancer using the Apache Zookeeper coordination service, with a manager process distributing tasks among several worker processes. The goal of this project was to explore coordination and communication within a distributed system using event-driven programming, where all message transmission was facilitated through Watcher objects and callback functions. Finally, we also implemented a time-slicing approach to ensure that more time-consuming tasks did not block other tasks from executing continuously for an excessive amount of time. The report outlines our design choices for the load balancer below.

2 Architecture and Design

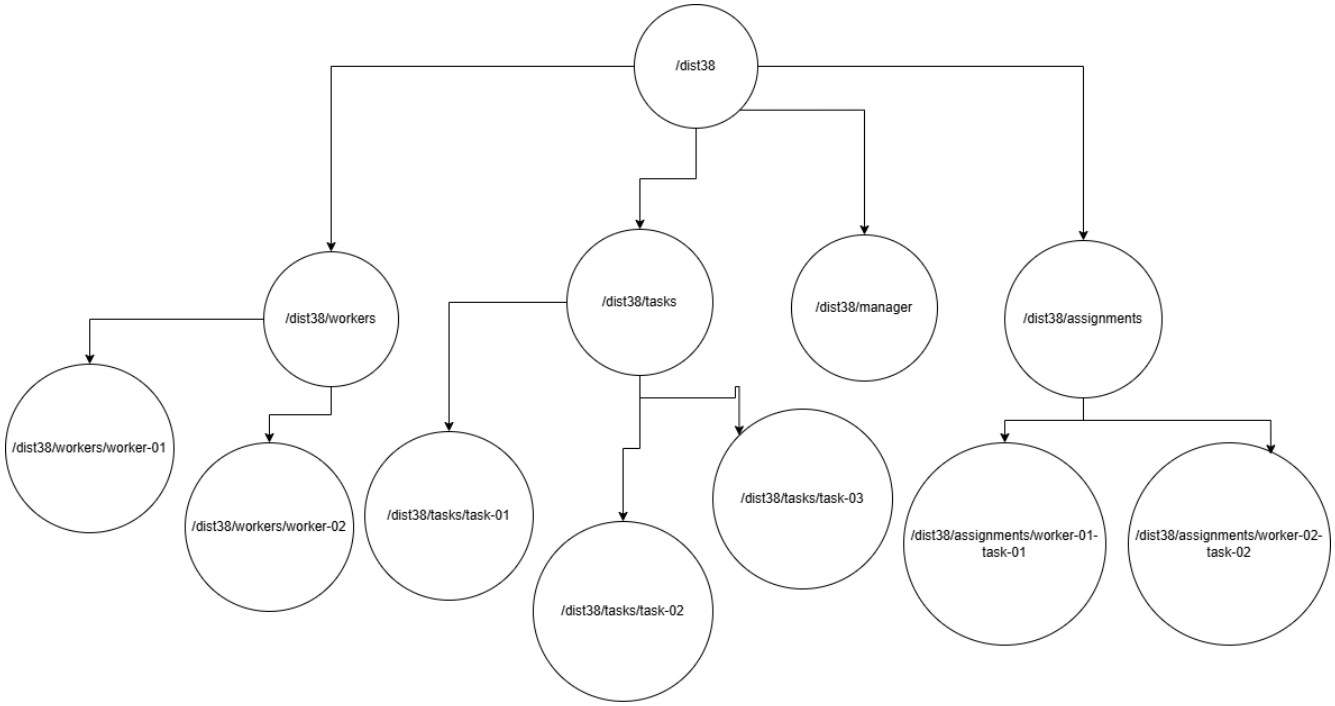


Figure 1: Apache Zookeeper Znode Hierarchy for Load Balancer (using two workers, three tasks)

At the top of our znode hierarchy, we have our root `/dist38` node, which has three persistent child nodes with the following path names: `/dist38/workers`, `/dist38/tasks`, and `/dist38/assignments`. `/dist38/workers` has sequential ephermal children znodes, each representing a worker currently being managed in the system by the manager process. Our reasoning to implement them as sequential ephermal nodes is that we anticipate that an arbitrary number of workers may be in the system at a given time, and thus the sequencing mechanism provided by the Apache Zookeeper library seemed reasonable to utilize. Since a worker exists only for the duration of a given system instance, it is reasonable to represent worker znodes as ephemeral. Ephemeral znodes capture this temporary role because they are automatically removed when the worker's session ends. There is also a `/dist38/tasks` znode, which has sequential persistent children znodes representing tasks to be executed. These children are created by the clients

as given in the client starter code. Finally, we decided to add a `/dist38/assignments` znode, to hold child znodes representing a worker and the task which is currently assigned to that worker. In other words, any worker whose name is contained within the name of these child znodes is currently busy with a task, and cannot process other tasks until it has finished the task, or an interrupt has occurred due to time slicing. The names of these child znodes are constructed as `worker_name-task_name` to represent `worker_name` being assigned (by the manager) to `task_name`. This naming scheme allows workers to view both the worker of a given assignment and the corresponding task of the assignment. These znodes also contain data representing the specific task to be executed, stored as a sequence of bytes. We decided to implement these child znodes as ephermal sequential znodes for similar reasons as for the worker znodes. However, given that the manager process creates the child znodes of `/dist38/assignments`, these child znodes are only deleted when the manager session with Apache Zookeeper ends (as the manager acts as a client to Zookeeper.) Because of this design, each worker znode deletes its own assignment child znode to trigger the manager's watch when it completes a task, rather than relying on the manager disconnecting to delete it. As a result, whether these assignment znodes are ephemeral or persistent is irrelevant.

3 Time Slicing Approach

To prevent long-running tasks from overwhelming worker resources, we implemented a preemptive time-slicing mechanism with a 1 second time slice.

3.1 Task Interruption and State Management

When a worker begins executing a task, it spawns a timer thread set to interrupt the worker thread upon the expiration of the time slice. Following interruption, the worker performs a checkpointing procedure:

1. **State Serialization:** The task's current execution state is serialized (e.g., the iteration count for the `MCPi` task).
2. **ZooKeeper Update:** The task's znode in ZooKeeper is updated with the serialized state.
3. **Assignment Deletion:** The worker deletes its active assignment from the assignment set.

This design allows the task to resume from its saved state when rescheduled, eliminating the need to restart computation from the beginning.

3.2 Interrupted Task Reassignment

The Manager component monitors the `/dist38/assignments` znode to detect interrupted tasks. A task is identified as interrupted and eligible for reassignment if it was **previously assigned**, **lacks a result znode**, and **has no active assignment**. Tasks that meet the three criteria are re-queued for reassignment to available workers.

3.3 Concurrency and Race Condition

To prevent race conditions, particularly those resulting from rapid ZooKeeper event propagation, we maintain a set that keeps track of which assignments each worker has actively begun processing. This prevents the creation of duplicate execution threads. We also ensure that each task runs in a dedicated thread to avoid blocking ZooKeeper's event processing loop.

3.4 Time Slice Selection

We determined the optimal time slice duration by testing values ranging from 100ms to 10 seconds. Our experiments indicated that system performance stabilized at approximately 500ms. Shorter intervals yielded diminishing returns due to the overhead associated with frequent interruptions, state serialization, and ZooKeeper communication. While performance remained comparable beyond 500ms, we selected **1 second** as the optimal time slice. This

duration provides the optimal balance among system responsiveness, fair resource allocation, and overall operational efficiency.

This choice ensures that multiple tasks can make incremental progress, even when certain tasks are computationally heavy.

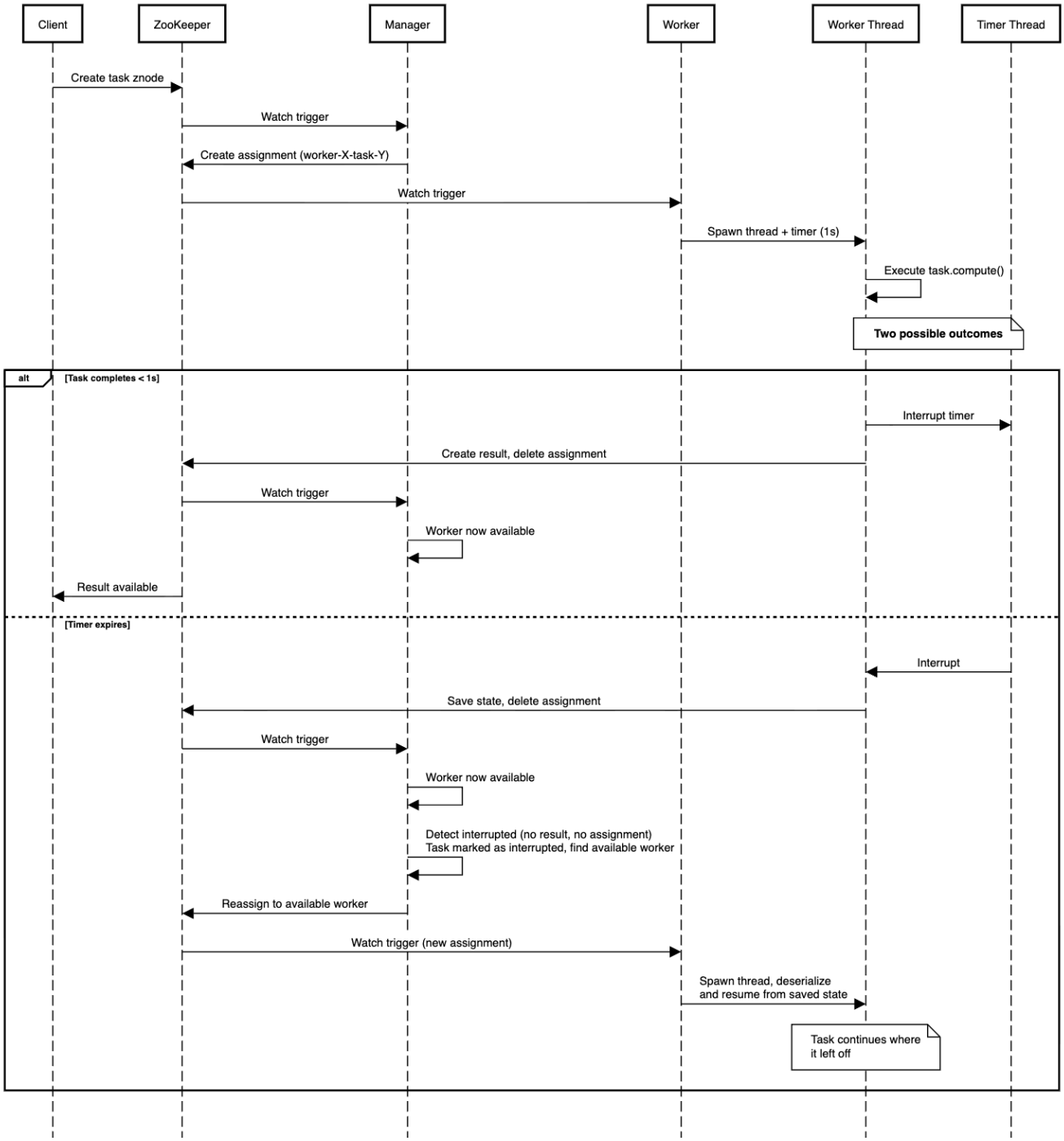


Figure 2: Time Slicing Sequence Diagram

4 Contributions

Ion was mainly responsible for implementing the load balancing logic. This included implementing data structures to track waiting tasks and available workers, and handling cases such as new workers joining after initialization. Karl was mainly responsible for implementing the time slicing approach, implementing the logic for thread interrupts and refining Ion's implementation to ensure the time slicing code was compatible (e.g. by adding extra conditions and data structures to check when tasks were fully completed.) However, we both reviewed and made changes to each others code, particularly to handle edge cases that the other person had missed. We used WhatsApp for communication to discuss new changes and the distribution of assignment tasks.